Estrutura Técnica do Projeto

Chama Aí – Plataforma de Serviços do Interior
Autor: Daniel Victor Bezerra | Versão 1.0 | Macro Entrega 2 – Projeto e Arquitetura

1. Repositório Git

Link do repositório	https://github.com/<usuario>/chama-ai
Visibilidade	Privado durante o desenvolvimento (por conter regras de segurança); acesso ao professor/orientador.
Branches	**main** (estável, protegida), **develop** (integração), **feature/<id>-<descrição>** e **fix/<descrição>**.
Fluxo	Branch por história de usuário, Pull Request para develop (com checklist), merge em main ao final de cada sprint com tag de versão (v0.1.0, v0.2.0...).
Commits	Conventional Commits em português (feat, fix, docs, test, refactor, chore).

2. Estrutura de pastas

```
chama-ai/
|-- README.md                    visão geral e como executar
|-- CONTRIBUTING.md              padrões de desenvolvimento
|-- docker-compose.yml           PostgreSQL local
|-- .env.example                 variáveis de ambiente (sem segredos)
|-- .gitignore
|-- .github/
|   |-- workflows/ci.yml         build e testes (GitHub Actions)
|   `-- pull_request_template.md
|-- docs/                        PDFs das macro entregas
|   `-- prototipos/              protótipos HTML (P01 a P11)
|-- frontend/                    React + TypeScript (Lovable) - README com regras
`-- backend/
    |-- pom.xml
    |-- Dockerfile
    `-- src/
        |-- main/
        |   |-- java/br/com/chamaai/
        |   |   |-- ChamaAiApplication.java
        |   |   |-- auth/           cadastro, login, JWT
        |   |   |-- usuario/        usuário e tipos
        |   |   |-- prestador/      perfil, serviços, disponibilidade, busca
        |   |   |-- documento/      envio e verificação do documento
        |   |   |-- agendamento/    solicitação, confirmação, cancelamento
        |   |   |-- admin/          painel do administrador
        |   |   |-- catalogo/       categorias e cidades
        |   |   `-- compartilhado/  segurança, CORS, erros, armazenamento
        |   `-- resources/
        |       |-- application.yml
        |       `-- db/migration/   V1__criar_tabelas.sql, V2__dados_iniciais.sql
        `-- test/                   testes (JUnit) e application-test.yml
```

3. Tecnologias e versões

Item	Versão / ferramenta	Uso
Java	21 (LTS)	Linguagem do back-end
Spring Boot	3.5.x	Framework (parent do Maven)
Maven	3.9+	Build e dependências
PostgreSQL	16	Banco de dados
Flyway	gerenciado pelo Spring Boot	Migrações do esquema
Node.js / Vite	LTS atual	Execução do front-end
React + TypeScript	gerado pelo Lovable	Front-end
Docker / Compose	atual	Banco local e imagem da API
Git / GitHub / GitHub Actions	—	Versionamento e integração contínua
Postman ou Swagger UI	—	Testes manuais da API

4. Dependências do back-end (pom.xml)

Dependência	Finalidade
spring-boot-starter-web	API REST (controllers, JSON)
spring-boot-starter-validation	Validação de entrada (Bean Validation)
spring-boot-starter-data-jpa	Acesso a dados com JPA/Hibernate
spring-boot-starter-security	Autenticação e autorização
spring-boot-starter-actuator	Endpoint de saúde (/actuator/health)
flyway-core e flyway-database-postgresql	Migrações do banco
postgresql (runtime)	Driver JDBC
jjwt-api, jjwt-impl, jjwt-jackson 0.12.6	Geração e validação de JWT
springdoc-openapi-starter-webmvc-ui 2.8.6	Swagger UI e contrato OpenAPI
lombok	Reduz código repetitivo nas entidades
spring-boot-starter-test, spring-security-test, h2 (test)	Testes automatizados

5. Configuração e variáveis de ambiente

Variável	Padrão (dev)	Descrição
DB_URL, DB_USER, DB_PASSWORD	localhost:5432/chamaai	Conexão com o PostgreSQL
JWT_SECRET	valor de desenvolvimento	Segredo do JWT; obrigatório trocar em produção (mín. 32 caracteres)
JWT_EXPIRACAO_MINUTOS	120	Validade do token
ARMAZENAMENTO_DIR	./armazenamento	Pasta privada dos documentos (fora do Git)
CORS_ORIGENS	http://localhost:5173	Domínios permitidos do front-end

6. Como executar

```
cp .env.example .env
docker compose up -d banco
cd backend
mvn spring-boot:run        # API: http://localhost:8080
                           # Swagger: http://localhost:8080/swagger-ui.html
mvn verify                 # compila e executa os testes
```

7. Padrões de desenvolvimento

Área	Padrão adotado
Código Java	Java 21; Google Java Style (4 espaços, até 120 colunas); nomes de domínio em português sem acento e sufixos técnicos em inglês (Controller, Service, Repository); pacote por funcionalidade.
Arquitetura	Controllers finos; regras de negócio nos Services; entidades JPA nunca expostas (usar DTOs em record); injeção por construtor.
API	Prefixo /api, JSON, recursos no plural; autenticação Bearer JWT; erros em Problem Details (RFC 7807); documentação OpenAPI.
Banco	Somente migrações Flyway versionadas (V3__descricao.sql); migração aplicada nunca é editada; ddl-auto=validate.
Testes	JUnit 5 e Mockito para regras (RN04 a RN07, RN12 a RN14); testes de endpoints críticos; H2 nos testes e PostgreSQL local para validar migrações.
Git	Branches feature/fix, Conventional Commits, Pull Request com checklist, main protegida.
Segurança	Segredos só em variáveis de ambiente; nada sensível no Git; documentos reais nunca em testes, no repositório ou em ferramentas de IA.
Uso de IA	Todo código gerado por Claude ou Lovable é lido, entendido e testado antes do commit; usos relevantes registrados no Plano de Utilização de IA.
Front-end	Consome somente a API Java via VITE_API_URL; sem Supabase; componentes reutilizáveis; layout mobile first.

8. Integração contínua e qualidade

- O fluxo do GitHub Actions (*.github/workflows/ci.yml*) executa *mvn verify* com Java 21 a cada push em main/develop e em Pull Requests.
- Definição de Pronto: critérios de aceitação atendidos, testes passando, código gerado por IA revisado, nenhum segredo ou documento real no commit, funcionalidade validada no celular.

- Evoluções futuras: análise estática (Checkstyle/SpotBugs), cobertura (JaCoCo) e implantação automática.

9. Estado inicial do repositório e próximos passos

Já incluído no esqueleto	Próximos passos (Sprint 1)
Estrutura de pastas e de pacotes; pom.xml com dependências; configuração por variáveis de ambiente.	Cadastro e login com BCrypt e JWT (US01, US02).
Migrações Flyway com o esquema completo e dados iniciais.	Filtro JWT, perfis de acesso e CORS.
SecurityConfig inicial e fatia de exemplo (GET /api/categorias e /api/cidades).	Tratamento global de erros (Problem Details).
Docker Compose do banco e Dockerfile da API.	Interface ArmazenamentoArquivos (implementação local).
CI, modelo de Pull Request, README e CONTRIBUTING.	Primeiro prompt do Lovable com base nos protótipos e no contrato OpenAPI.